

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192324035
Name	Shri Shanmuga Priya K

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Shri Shanmuga Priya K (192324035) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS-Based Storage Solution for a Media Platform

Problem

A media platform stores large amounts of videos, audio files, images, thumbnails and metadata. The solution must satisfy:

- Data availability > 99.9%
- Replication factor ≤ 3
- Cost-efficient storage growth

Solution

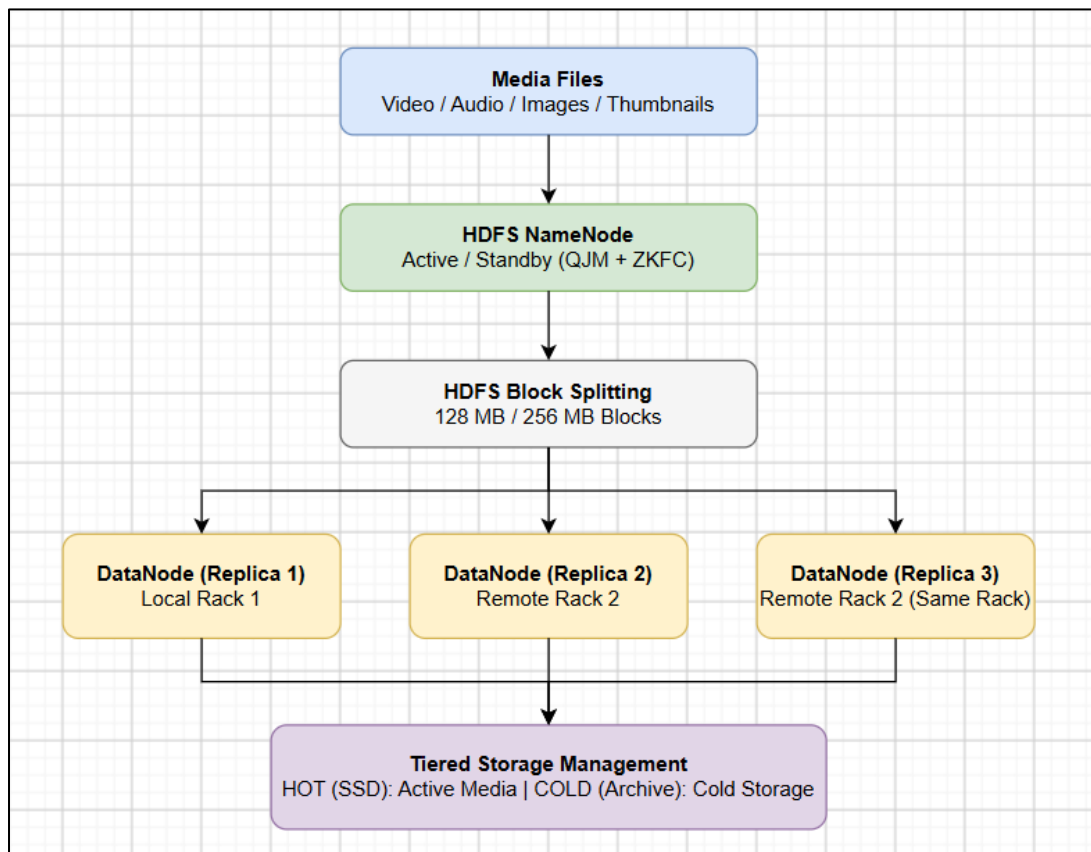
An HDFS-based architecture can satisfy these requirements using a replication factor of 3, rack-aware placement and efficient storage management.

HDFS divides large files into blocks and distributes them across DataNodes. A replication factor of 3 provides multiple copies of important data while remaining within the specified limit.

The NameNode manages file metadata, while DataNodes store the actual media blocks. NameNode high availability can be implemented using an Active/Standby NameNode configuration.

For cost efficiency, frequently accessed media can remain on faster storage, while older or rarely accessed content can be moved to lower-cost storage using storage policies.

Flowchart



Justification

Availability: Three replicas allow data to remain accessible if a DataNode fails. Rack-aware placement prevents all replicas from being placed on the same physical rack.

Replication constraint: The replication factor is set to **3**, which is the maximum allowed.

Cost efficiency: Increasing replication beyond 3 is avoided. Older and less frequently accessed data can be moved to cheaper storage.

Scalability: New DataNodes can be added when media storage grows.

Conclusion

A 3-replica, rack-aware HDFS architecture with NameNode high availability and storage-tier management provides high availability while controlling storage cost.

2. MapReduce Workflow Within a 20-Minute Batch Window

Problem

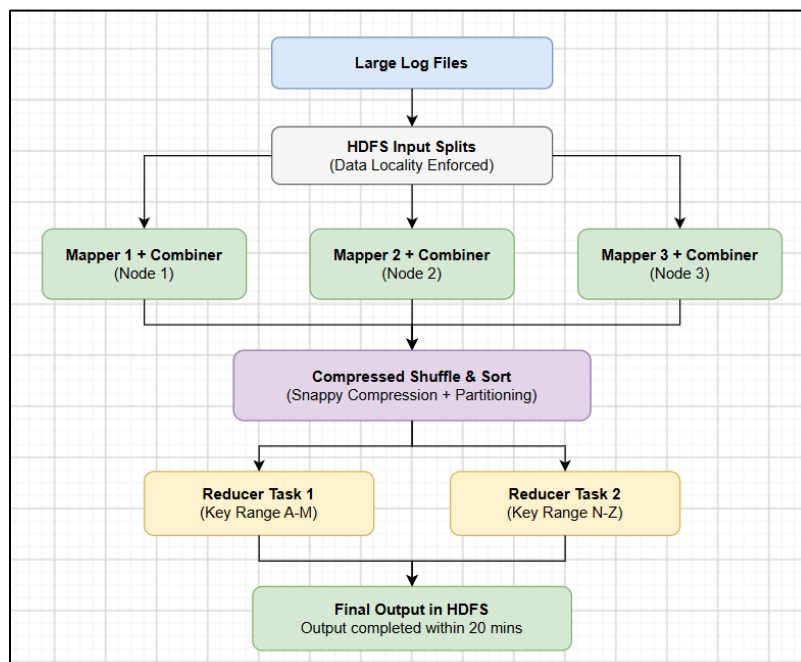
A MapReduce job must process large log data within **20 minutes**, but only a limited number of cluster nodes are available.

Solution

The log files should first be divided into appropriate HDFS blocks. Map tasks process these blocks in parallel. The intermediate results are then shuffled and sorted before being processed by Reduce tasks.

The number of mapper and reducer tasks should be selected according to the available CPU, memory and number of nodes. A simple log-processing example could calculate the number of requests, errors or users per hour.

Flowchart



Task Distribution

Suppose the cluster has 4 worker nodes:

- Each node runs mapper tasks for local HDFS blocks.
- Multiple mapper tasks can execute sequentially or concurrently depending on available resources.
- Reducers process different key ranges in parallel.

Methods to Meet the 20-Minute Constraint

1. Use data locality so computation occurs close to the data.
2. Select an appropriate number of mapper tasks.
3. Use sufficient reducer parallelism without overloading nodes.
4. Compress intermediate Map output.
5. Use combiners when applicable to reduce network traffic.
6. Avoid unnecessary data movement.
7. Monitor slow tasks and handle stragglers.

Conclusion

Parallel execution of Map tasks followed by distributed Reduce tasks allows the workload to be completed within the **20-minute batch window**, despite limited nodes.

3. YARN Scheduler Policy

Problem

A YARN cluster runs:

- ETL jobs
- Reporting jobs
- Machine learning jobs

The scheduler must provide fairness, give priority to ETL, and recover from node failures.

Solution

A Capacity Scheduler or appropriately configured Fair Scheduler-style policy can be used to divide resources among workloads.

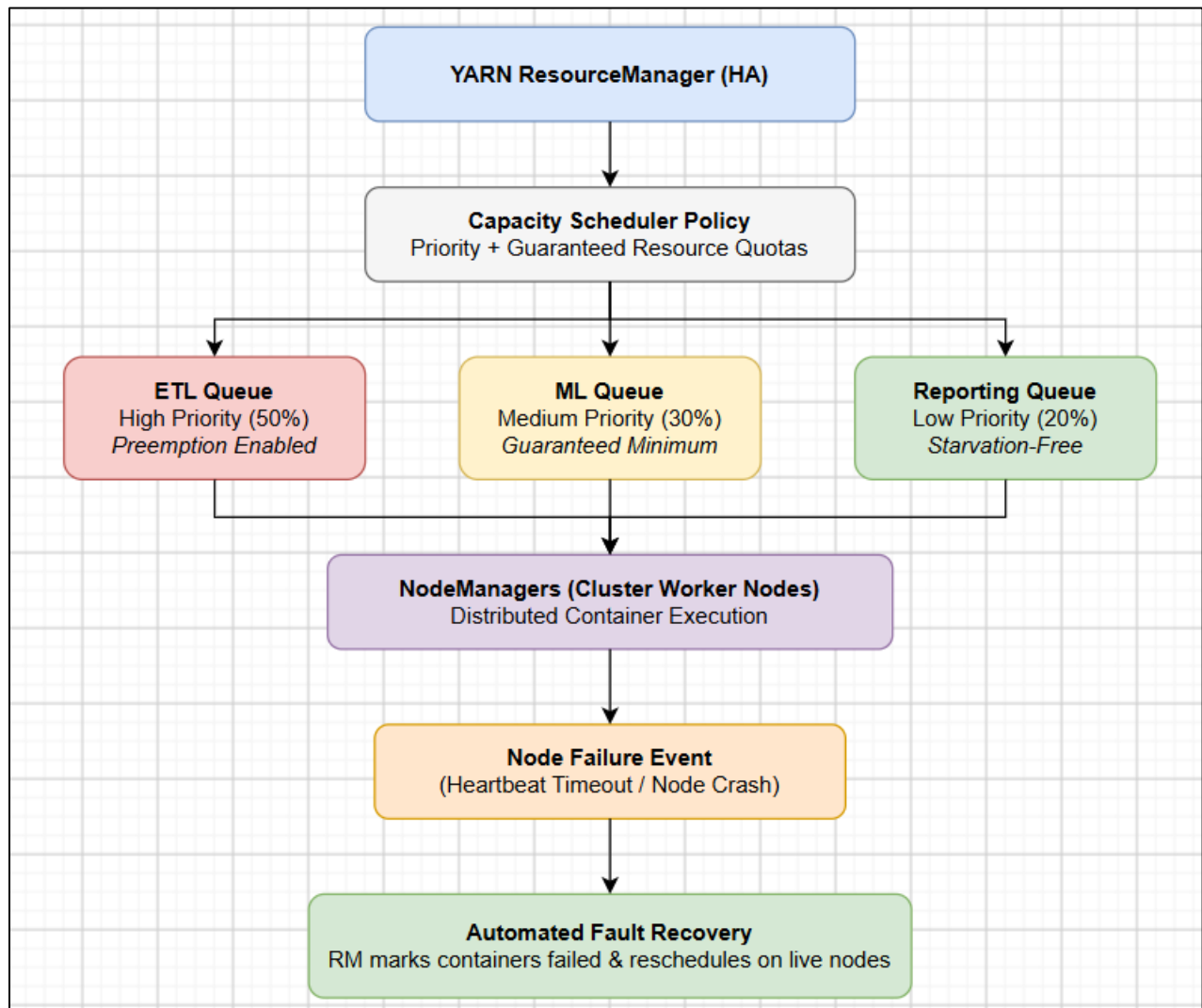
ETL should receive higher priority because it is the main data-preparation workload. Reporting and ML jobs should still receive guaranteed resources so that they are not completely starved.

Proposed Policy

Workload	Priority	Resource Allocation
ETL	High	50%
Reporting	Medium	20%
Machine Learning	Medium	30%

These percentages can be adjusted according to actual workload requirements.

Flowchart



Recovery

If a NodeManager fails:

1. ResourceManager detects the failed node.
2. Running containers on that node are considered failed.

3. Tasks are rescheduled on healthy nodes.
4. The scheduler reallocates available resources.
5. Failed jobs continue from recoverable checkpoints where supported.

Conclusion

The policy provides ETL priority without completely starving reporting and ML, while YARN's resource management and task rescheduling provide recovery from node failures.

4. Enterprise SAN, NAS and Database Ingestion

Problem

An enterprise needs to ingest data from: SAN, NAS, Operational databases

The architecture must provide reliable backup and minimal duplication.

Solution

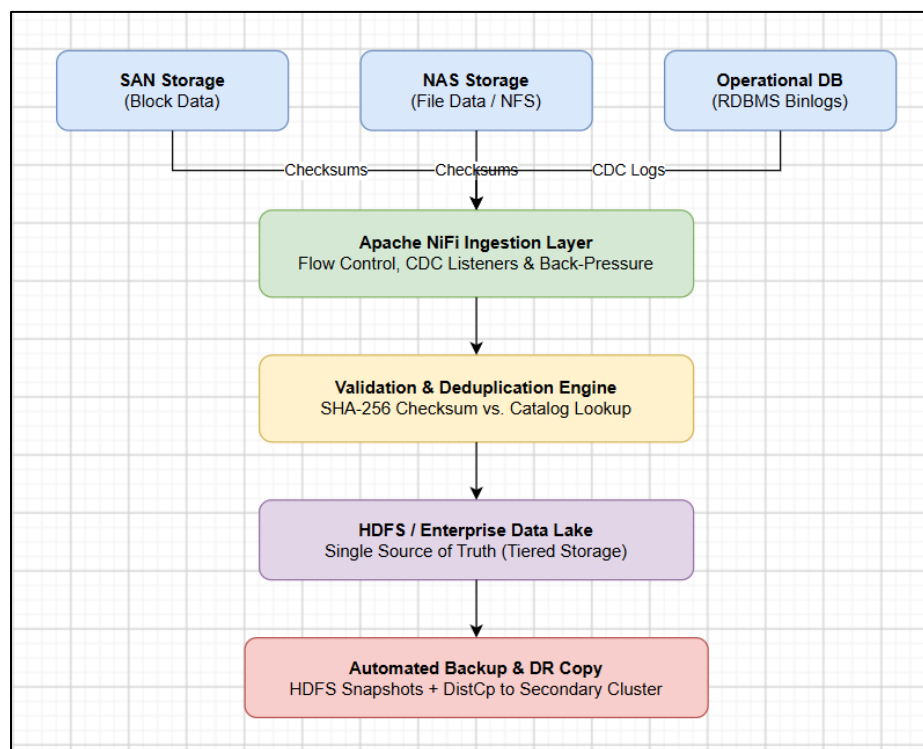
A centralized ingestion layer can be created using Apache NiFi or another ETL platform.

NiFi can connect to different sources, route data, transform it and deliver it to the big data platform.

For databases, CDC (Change Data Capture) can be used to capture only changed records rather than repeatedly copying the complete database.

For SAN/NAS files, metadata and checksums can help identify files that have already been ingested.

Flowchart



Component Justification

Apache NiFi:

Provides visual data-flow management, routing, transformation, monitoring and back-pressure.

CDC:

Captures only changed database records and reduces duplicate ingestion.

Checksums:

Can identify whether a file has already been ingested.

HDFS/Data Lake:

Provides distributed storage and replication.

Backup:

Important datasets can be replicated to a separate storage system or disaster-recovery location.

Minimal Duplication

A metadata catalog can maintain:

File ID

Source

Timestamp

Checksum

Destination

Processing Status

Before ingesting a file, its checksum can be compared with previously processed files.

Conclusion

An architecture based on NiFi + CDC + validation/deduplication + HDFS/data lake + backup storage provides reliable ingestion while minimizing unnecessary duplication.

5. End-to-End Hadoop Ecosystem Solution

Problem

The Hadoop ecosystem must satisfy three main constraints:

1. Secure data movement
2. High availability
3. Integration with Apache NiFi or ETL pipelines

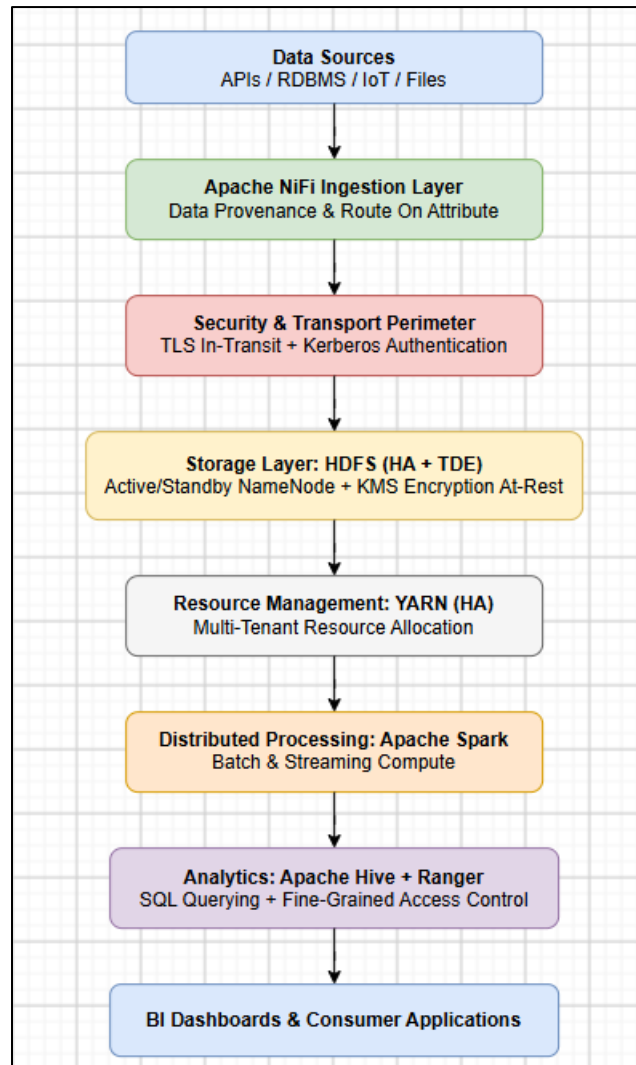
Solution

Apache NiFi can be used as the data ingestion layer. It collects data from databases, files, APIs and other systems.

The data can then be transferred securely into HDFS using encryption and authentication mechanisms.

HDFS provides distributed storage with replication, while YARN manages cluster resources. Spark can be used for large-scale processing.

Architecture Flowchart



How the Constraints Are Satisfied

1. Secure Data Movement

Data entering the platform can be protected using:

- TLS encryption
- Authentication
- Access control

- Kerberos in Hadoop environments
- Encryption at rest where required

Thus, unauthorized users cannot easily access data during transmission or storage.

2. High Availability

HDFS can use data replication to keep multiple copies of blocks.

The Hadoop control plane can use NameNode High Availability, with active and standby components.

3. NiFi Integration

NiFi can perform:

- Data ingestion
- Routing
- Transformation
- Validation
- Monitoring
- Back-pressure management

It can send processed data into HDFS or other Hadoop components.

4. Processing

Spark can perform large-scale processing, while Hive provides SQL-based analytics.

Conclusion

The complete architecture uses NiFi → secure transfer → HDFS → YARN → Spark/MapReduce → Hive → BI. HDFS replication and NameNode high availability provide resilience, while encryption and authentication protect data movement.